Quantum Fault-Zone Partitioning of the Costa Rican Power Grid

A Max-Cut / QAOA Study — Quantathon Challenge #1

Kraken Team — Gabriel Alba Romero, Juan C. Lara, Jonathan Gomez Barrios

July 24, 2026

1 Problem framing

Physical problem. A transmission grid must be split into self-sufficient *fault zones*: when a fault occurs, protection relays open a small set of tie-lines so the faulted region can be isolated *without* severing the lines that carry the most power or that hold the grid together. A good split therefore (i) cuts as little critical transmission capacity as possible and (ii) leaves generation on *both* sides so neither zone goes dark. We model this as a **graph-partitioning / Max-Cut** problem on the Costa Rican national grid and solve it both classically and with QAOA.

Graph model We snapshot two open ICE ArcGIS layers — *Subestaciones* (substations) and *Líneas de Transmisión* (transmission lines) — into a static, reproducible dataset and build a weighted undirected graph $G = (V, E)$. Substations become nodes; each line's `Circuito` field ("SubA–SubB") yields an edge. Parallel lines are collapsed (weights summed, highest voltage kept), and endpoints with no matching substation (international ties, SIEPAC, industrial loads) are marked as *border* nodes. The national graph has 75 nodes and 97 edges. Because exact solutions and today's emulators are limited to a modest qubit count, we deterministically extract a connected sub-region — **Guanacaste North**, $n = 9$ substations, 9 lines, 1 independent cycle, 7 of which host generation (max plant 306.2 MW).

Edge weights Every edge weight measures *how costly it is to cut that line.* Weight schemes are a registry of pure functions `fn(voltage, length, gens_u, gens_v)`; the project default, `generation_inverted`, is generator-aware and sign-inverted so that the *most critical* lines carry the *largest positive* weight ($w_{\max} = 16.106$ for this sub-grid). This lets a *minimize-cut* objective directly express "protect the critical lines."

Pipeline. The project is a linear, reproducible flow from raw ICE data to a quantum-ready cost Hamiltonian and a classical/quantum comparison (Fig. 1). Task A produces the weighted sub-graph, Task B the QUBO / Ising / cost Hamiltonian, Task C the QAOA solver; classical baselines score the *same* operator for a fair comparison.

2 Classical formulation and baselines

2.1 QUBO / Ising / cost Hamiltonian

Each node gets a binary variable $x_i \in \{0, 1\}$ labelling its fault zone. The weight of the cut is

$$\mathrm{Cut}(x) = \sum_{(i,j)\in E} w_{ij}\,(x_i + x_j - 2x_i x_j),$$

where the bracket is 1 exactly when the edge is cut. We *minimize* $\mathrm{Cut}(x)$ (not maximize): with the inverted-generation weights, cutting a critical line is expensive, so the boundary settles on the cheapest tie-lines. Two **quadratic** (ancilla-free) penalties, registered like the weight schemes, encode the operational constraints:

- *Generator spread* $P_{\text{gen}} \sum_{\text{gen pairs}} (1 - \mathrm{cut}_{ij})$ — keeps generation on both sides; symmetric, so it excludes both trivial all-same partitions. $P_{\text{gen}} = 0.5\, w_{\max}$ per pair.
- *Balance* $\lambda(\sum_i x_i - n/2)^2$ — discourages lopsided cuts. $\lambda = 0.15\, w_{\max}$.

The accumulated QUBO $\mathrm{cost}(x) = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j + \text{offset}$ is mapped to spins via $x_i = (1 - z_i)/2$, giving the diagonal **cost Hamiltonian**

$$H_C = \text{offset} \cdot I + \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j.$$

Because every term is at most two-local Z, H_C is diagonal in the computational basis: its ground state *is* the optimal fault-zone partition, and `QUBO.energy`, `CostHamiltonian.energy`, and the brute-force enumerator agree bit-for-bit. This single operator is what *every* solver — classical and quantum — is scored on (Fig. 3).

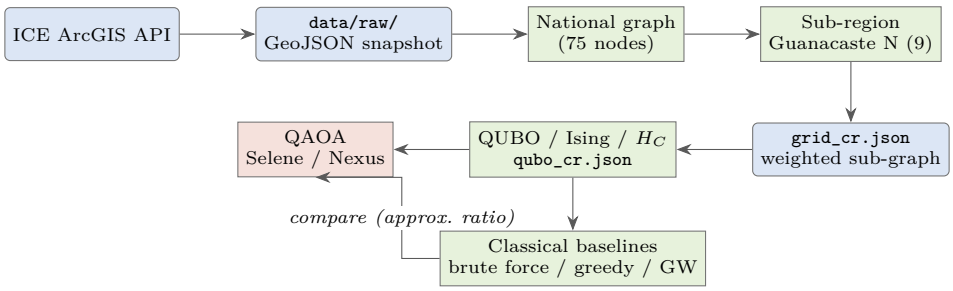
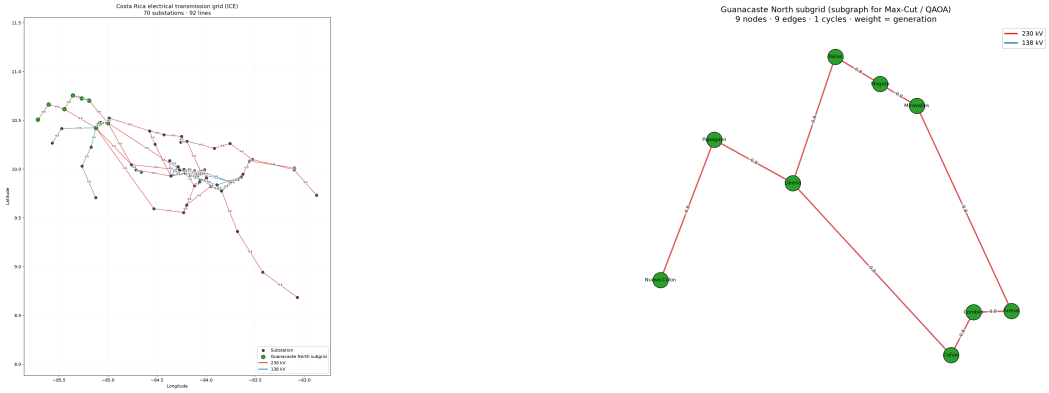


Figure 1: End-to-end pipeline. Blue = data artifacts, green = classical processing, red = quantum. Only the static snapshot is ever read downstream, so every run is reproducible.



(a) National grid; the Guanacaste-North sub-grid highlighted in green.

(b) The extracted 9-node sub-region (9 lines, 1 cycle) solved by Max-Cut / QAOA.

Figure 2: From the full ICE transmission grid (Task A) to the deterministic connected sub-region used throughout the study.

2.2 Classical baselines

All classical solvers optimize the *same* objective by running on `augmented_ising_graph(`H_C`)`, a weighted Max-Cut graph (a FIELD node carrying the fields h_i plus J_{ij} edges) whose maximum cut equals minimizing $\langle H_C \rangle$; partitions map back with `bits_from_partition`.

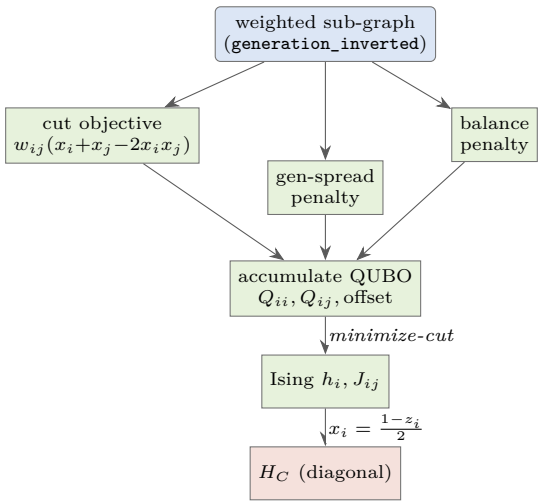
- **Brute force (exact reference).** A single vectorized enumerator (`enumerate_cut_spectrum`) sweeps all 2^n assignments in NumPy chunks, pinning one node (global-flip symmetry) so only 2^{n-1} are visited and peak memory stays proportional to the chunk. It yields the exact spectrum bounds $(e_{\min}, e_{\max})$ that anchor every score; a 300 s timeout falls back to a Goemans–Williamson proxy beyond $\sim$26 qubits.
- **Greedy (heuristic lower bar).** Visits nodes in a seeded random order, placing each on the side that maximizes immediate cut gain. As a sampler it runs `n_shots` independent seeded restarts — the same sample budget as QAOA shots.
- **Goemans–Williamson (strongest baseline).** Solves the SDP relaxation $\max \sum W_{ij}(1 - X_{ij})/2$ s.t. $\mathrm{diag}(X) = 1$, $X \succeq 0$ (cvxpy + SCS) *once*, then performs `n_shots` random-hyperplane roundings, carrying the classic 0.878 guarantee for non-negative weights.

Every solver produces bit assignments scored by the **offset-robust approximation ratio** $r = (e_{\max} - E)/(e_{\max} - e_{\min})$, so $r = 1$ is the classical optimum and different signs/offsets are comparable (Fig. 4).

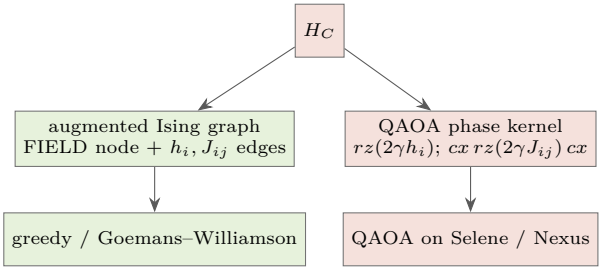
3 Quantum implementation

We solve the fault-zone problem with the **Quantum Approximate Optimization Algorithm (QAOA)**, written in **Guppy 0.21** and executed on the **Selene** emulator (locally) or the **Quantinuum Nexus** hosted emulator (Helios). The number of qubits (9), the operator H_C, and even the *minimize* direction are fixed by the QUBO — not free choices.

Weighted phase/mixer kernel. Because H_C is weighted and carries single-Z fields, we cannot reuse the unweighted Max-Cut example kernel. Following the diagonal-H_C mapping, one QAOA layer applies



(a) QUBO formulation (Task B).



(b) One H_C, two consumers: the augmented Max-Cut graph feeds the classical baselines, the phase kernel feeds QAOA.

Figure 3: From weighted graph to a single diagonal cost Hamiltonian scored by both families of solvers.

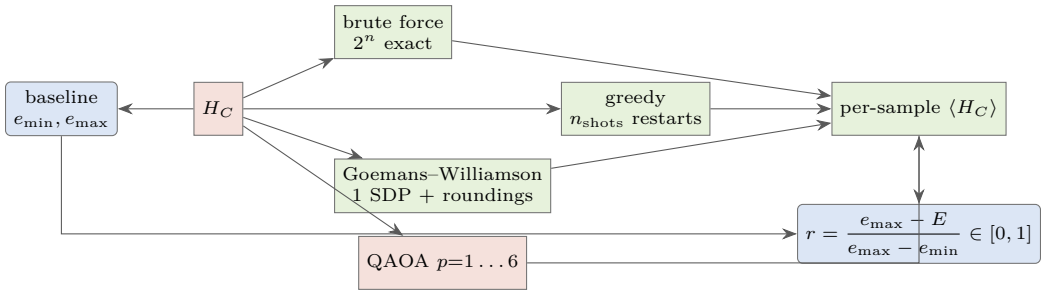


Figure 4: Every optimizer scores the same H_C; the brute-force spectrum bounds turn each energy into a comparable approximation ratio.

Term	Circuit fragment
$h_i Z_i$ (field)	$rz(2\gamma h_i,\, i)$
$J_{ij} Z_i Z_j$ (coupling)	$cx(i,j);\ rz(2\gamma J_{ij},\, j);\ cx(i,j)$
mixer	$rx(2\beta,\, i)$ per qubit
offset $\cdot\, I$	global phase — ignored

Coefficients are baked in as `comptime` constants. A Guppy 0.21 subtlety: `angle(x)` means x *half-turns* ($x\pi$ rad), so each $2h_i$ (and the mixer's 2) is folded with a $1/\pi$ factor at compile time to keep the radian convention while γ, β stay plain multipliers. Empty term lists are padded with a zero-coefficient placeholder ($rz(0)$ is identity) because Guppy cannot type-infer an empty `comptime` list.

Variational loop. Energy is the true expectation $\langle H_C \rangle$ from `CostHamiltonian.energy` weighted by shot probabilities. **SciPy COBYLA** minimizes it over the flattened $[\gamma(p), \beta(p)]$ parameters; a fixed seed makes every emulator evaluation deterministic. The kernel is backend-agnostic: it compiles to HUGR and runs on local Selene (`main.emulator(...).run()`) or Nexus (`upload`→`start_execute_job`→`wait_for`), which return the *same* `QsysResult` type, so decoding is identical. For $n \leq 12$ the exact ground state (`brute_force_ground_state`) and spectrum bounds provide the reference for the approximation ratio.

4 Results

Initially, a small smoke test was run to ensure that with a small graph we would have a solution that makes sense. The result from this small run was the graph detailed in Figure 5 5, which for this small dataset is the best cut possible. This signal indicates that the QAOA optimizer implemented can find good solutions to these types of issues.

The real question then becomes if it can generate them reliably?

To understand if the optimizer can generate repeatable results a statistical analysis over several runs was run. At the same time, we ran the 3 classical benchmarks to be able to compare QAOA to standard optimization practices.

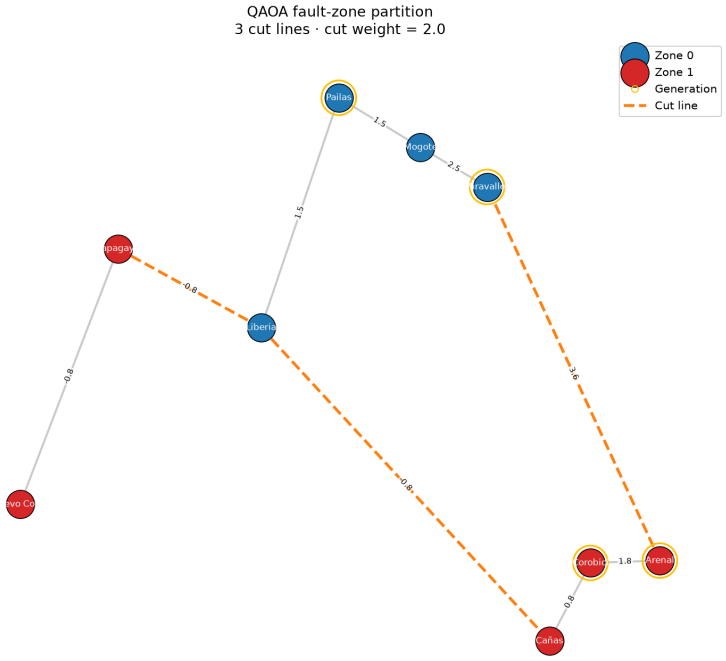


Figure 5: Fault-zone partition recovered by QAOA on the sub-grid: nodes coloured by zone, generation-bearing substations ringed, and the cut (protected boundary) lines dashed.

Table 1: Comparison on the Guanacaste-North instance ($n = 9$, $e_{\min} = 4.96$, $e_{\max} = 21.54$). Mean ± std over 3 replicate runs, 1000 shots each. r = approximation ratio (1 = classical optimum). Best-of-run r shown separately from the mean-over-samples r.

Optimizer	Best energy	Mean r	Best r	MSE vs. opt.	Time (s)
Goemans–Williamson	4.96 ± 0.00	0.962 ± 0.001	1.000 ± 0.000	0.64 ± 0.02	1.9 ± 1.3
Greedy	4.96 ± 0.00	0.939 ± 0.001	1.000 ± 0.000	1.64 ± 0.03	2.0 ± 1.3
QAOA $p = 1$	5.15 ± 0.26	0.676 ± 0.004	0.989 ± 0.016	39.1 ± 0.7	170 ± 13
QAOA $p = 3$	5.15 ± 0.26	0.733 ± 0.075	0.989 ± 0.016	29.4 ± 12.3	290 ± 4

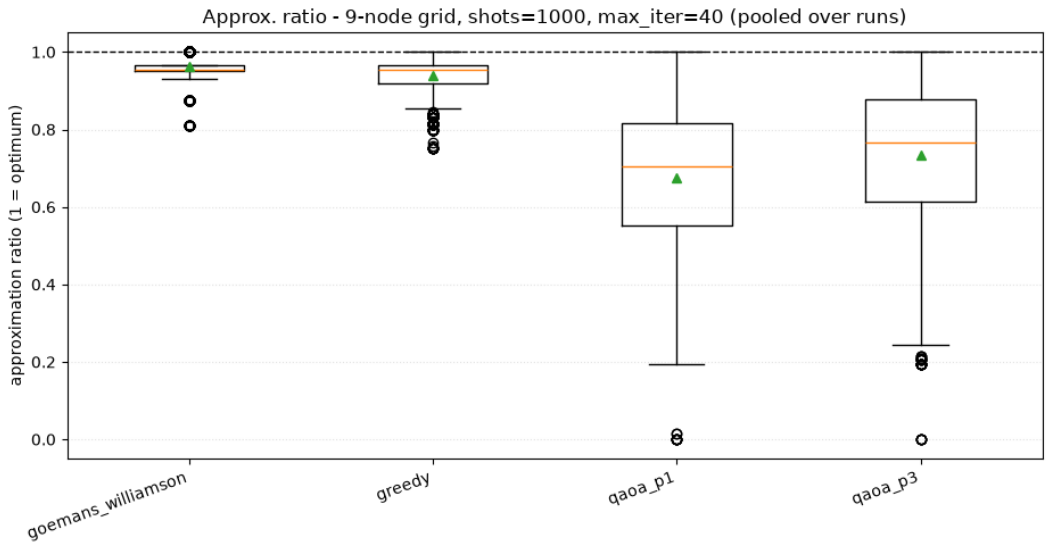
The Table 1 results are expected, for the parameters chosen GW and the greedy algorithm show far superior performance both in terms of time and in terms of the approximation ratio. From the literature we know that the expected 'r' is around 0.6 so the performance achieved is on par with the results achieved by other implementations in the field.

Reading the table. The exact optimum ($e_{\min} = 4.96$) is reached by both classical baselines and, in its best shot, by QAOA (best $r = 0.989$–1.0). The *mean*-over-samples ratio separates the methods: Goemans–Williamson concentrates near the optimum (0.962), greedy trails slightly (0.939), and shallow QAOA has a broader shot distribution (0.676 at $p = 1$, rising to 0.733 at $p = 3$) — deeper circuits sharpen the distribution toward the optimum, at higher runtime. Error bars come from the 3 replicate runs (independent seeds); the classical baselines are essentially deterministic on this instance while QAOA shows the expected run-to-run variance.

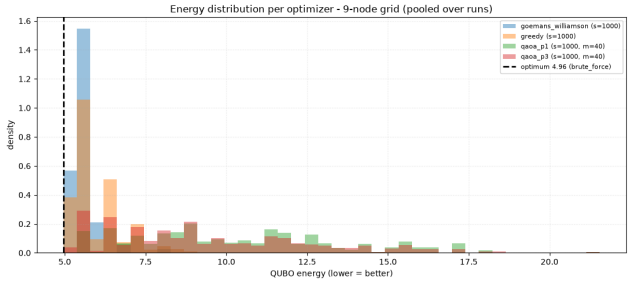
Both Figure 6a and Figure 6b show a lot of different aspects of the results achieved. In the first place we can observe the dramatic difference in the distribution of results. As seen, both QAOA algorithms have a very broad distribution which indicates that the results from this algorithm are not as consistent as the classical algorithms. This is most likely due to noise in the model.

Additionally, it is also notable that both the median and the mean of the distribution for QAOA is lower than for the classical algorithms. This is expected for such a small graph, since greedy can easily optimize this size of a dataset. The expectation would be for the greedy algorithm to quickly fall off as the problem gets bigger.

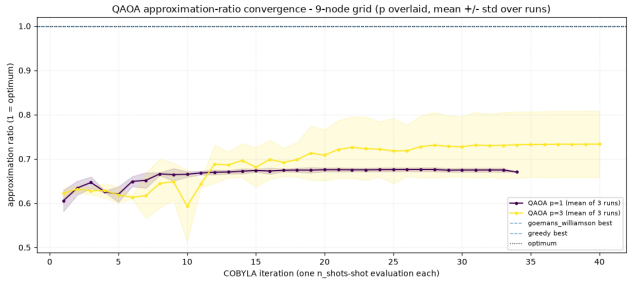
Finally, in terms of the convergence of the algorithm Figure 6c shows how the approximation ratio behaved as the optimization iterations were running. In this graph it is easy to see that when p = 3, the mean of the results definitely sits higher than when p = 1, confirming the expected behavior that with more layers, it is able to find better solutions. Even then, the standard deviation is larger for the 3 layer solution indicating that the introduction of more layers also adds more noise to the solution. What this means is that there is most likely



(a) Approximation-ratio distribution per optimizer (box = interquartile range, whiskers, green triangle = mean); 1.0 is the classical optimum.



(b) Pooled per-shot energy distributions; dashed line = brute-force optimum 4.96.



(c) QAOA approximation-ratio convergence vs. COBYLA iteration, mean ± std over the 3 replicate runs.

Figure 6: Results on the Guanacaste-North instance ($n = 9$, 1000 shots, 3 replicate runs). Error bars/spreads come from the replicate runs and the shot distributions.

a divergence point where noise takes over and the standard deviation becomes unacceptably large.

5 Conclusions

From the development of this project it can be concluded that the QAOA optimizer can solve max-cut problems with an approximation ratio close to 0.6, which is lower than the baseline classical algorithms for these same type of problems. What this indicates is that the technology is still very immature and there is still a lot of room to grow to catch up and eventually surpass the classical baselines.

6 Honest limitations

During the execution of the hackathon there were a few limitations that were found. In the first place, from our baseline case of 9 nodes to the max possible 26 qubits QAOA scales in a very steep fashion, making it unfeasible to be able to run as many evaluations as we wanted. Additionally, the speed of the emulators was a huge bottleneck when trying to run optimizations since a single cycle was needed for every optimization iteration, which meant we had to end up defaulting to using Selene local emulation instead of Nexus' backends. There is a small caveat here, where our web-app does run in the Nexus backend. Likewise, we also faced a big limitation in terms of time. Since none of the team members had any physics or quantum computing background, most of the first day had to be spent in gaining a better understanding of the QAOA optimizer and what the problem was at it's core, which in the end meant less time for running evaluations.

7 Future Work

To further develop this solution more things can be done. These are some of the possible paths that may be taken to achieve even better results:

1. The weight calculation for th transmission lines should be fine-tuned to better represent the importance of a line
2. The penalty coefficients in the QUBO calculation must also be fine tuned further to be able to get better approximation ratios.

3. The implementation of a weighted AND directed QUBO representation may provide even beter solutions since it may be able to take into account transmission lines direction into the weight calculation.
4. Use different variations of the QAOA optimizer is also worth exploring such as warm-start or recursive QAOA may present better performance.
5. Noise reduction algorithms could also increase the performance of the optimizer
6. Error Correction or Error Detection algorithms may also help in achieving a better performance with the tradeoff of considerably longer running times for the algorithm.